

Práctica 5

1. Se requiere modelar un puente de un único sentido que soporta hasta 5 unidades de peso. El peso de los vehículos depende del tipo: cada auto pesa 1 unidad, cada camioneta pesa 2 unidades y cada camión 3 unidades. Suponga que hay una cantidad innumerable de vehículos (A autos, B camionetas y C camiones). Analice el problema y defina qué tareas, recursos y sincronizaciones serán necesarios/convenientes para resolverlo.

a. Realice la solución suponiendo que todos los vehículos tienen la misma prioridad.

Procedure PasoPuenteA is

Task puente is

Entry CruzarAuto();
Entry CruzarCamioneta();
Entry CruzarCamion();
Entry SalirAuto();
Entry SalirCamioneta();
Entry SalirCamion();

End puente;

Task Type auto

End auto;

Task Type camioneta

End camioneta;

Task Type camion

End camion;

arrAutos: array (1..A) of auto;

arrCamionetas: array (1..B) of camioneta;

arrCamiones: array (1..C) of camion;

Task Body Auto is

Begin

 Puenete.CruzarAuto();

 CruzarPuenete();

 Puenete.SalirAuto();

End Auto;

Task Body Camioneta is

Begin

 Puenete.CruzarCamioneta();

 CruzarPuenete();

 Puenete.SalirCamioneta();

End Camioneta;

Task Body Camion is

Begin

 Puenete.CruzarCamion();

 CruzarPuenete();

 Puenete.SalirCamion();

End Camion;

Task Body Puenete is

 peso_total: int;

Begin

 peso_total = 0;

 loop

 SELECT

 // Primero darle prioridad a los vehiculos que salen

 ACCEPT SalirAuto

```

        peso_total -= 1;
    end SalirAuto
OR
    ACCEPT SalirCamioneta
        peso_total -= 2;
    end SalirCamioneta
OR
    ACCEPT SalirCamion
        peso_total -= 3;
    end SalirCamion
// Y luego encargarse de los que entran
OR
    WHEN (peso_total + 1 <= 5) ⇒ ACCEPT CruzarAuto
        peso_total += 1;
    end CruzarAuto
OR
    WHEN (peso_total + 2 <= 5) ⇒ ACCEPT CruzarCamioneta
        peso_total += 2;
    end CruzarCamioneta
OR
    WHEN (peso_total + 3 <= 5) ⇒ ACCEPT CruzarCamion
        peso_total += 3;
    end CruzarCamion
end loop;
End Puente;
Begin
    null;
End PasoPuenteA;

```

[Corregido]

b. Modifique la solución para que tengan mayor prioridad los camiones que el resto de los vehículos

```

Procedure PasoPuenteB is
    Task puente is

```

```

    Entry CruzarAuto();
    Entry CruzarCamioneta();
    Entry CruzarCamion();
    Entry SalirAuto();
    Entry SalirCamioneta();
    Entry SalirCamion();
End puente;

Task Type auto
End auto;

Task Type camioneta
End camioneta;

Task Type camion
End camion;

arrAutos: array (1..A) of auto;
arrCamionetas: array (1..B) of camioneta;
arrCamiones: array (1..C) of camion;

Task Body Auto is
Begin
    Puente.CruzarAuto();
    CruzarPuente();
    Puente.SalirAuto();
End Auto;

Task Body Camioneta is
Begin
    Puente.CruzarCamioneta();
    CruzarPuente();
    Puente.SalirCamioneta();
End Camioneta;

Task Body Camion is

```

```

Begin
    Puente.CruzarCamion();
    CruzarPuente();
    Puente.SalirCamion();
End Camion;

Task Body Puente is
    peso_total: int;
Begin
    peso_total = 0;
    loop
        SELECT
            // Primero darle prioridad a los camiones
            ACCEPT SalirCamion
                peso_total -= 3;
            end SalirCamion
        OR
            WHEN (peso_total + 3 <= 5) ⇒ ACCEPT CruzarCamion
                peso_total += 3;
            end CruzarCamion
        // Luego encargarse de los otros que salen
        OR
            ACCEPT SalirCamioneta
                peso_total -= 2;
            end SalirCamioneta
        OR
            ACCEPT SalirAuto
                peso_total -= 1;
            end SalirAuto
        // Y luego de los otros que entran
        OR
            WHEN (CruzarCamion'count = 0 && peso_total + 2 <= 5) ⇒
ACCEPT CruzarCamioneta
                peso_total += 2;
            end CruzarCamioneta
        OR

```

```

                WHEN (CruzarCamion'count = 0 && peso_total + 1 <= 5) =>
ACCEPT CruzarAuto
                peso_total += 1;
                end CruzarAuto
            end loop;
        End Puente;
Begin
    null;
End PasoPuenteB;

```

[[Consultar](#)]

2. Se quiere modelar el funcionamiento de un banco, al cual llegan clientes que deben realizar un pago y retirar un comprobante. Existe un único empleado en el banco, el cual atiende de acuerdo con el orden de llegada.

a. Implemente una solución donde los clientes llegan y se retiran sólo después de haber sido atendidos

```

Procedure BancoA is
    Task empleado is
        Entry pagar(com: OUT comprobante);
    End empleado;

    Task Type cliente
    End cliente;

    arrClientes: array (1..C) of cliente;

    Task Body cliente is
        Comprobante com;
    Begin
        Empleado.pagar(com);
    End cliente;
End BancoA;

```

```

End;

Task Body empleado is
    Comprobante com;
Begin
    loop
        ACCEPT pagar(com: OUT comprobante)
            com = generarComprobante();
        End pagar;
    end loop;
End empleado;
Begin
    null;
End BancoA;

```

[Corregido]

b. Implemente una solución donde los clientes se retiran si esperan más de 10 minutos para realizar el pago

```

Procedure BancoB is
    Task empleado is
        Entry pagar(com: OUT comprobante);
    End empleado;

    Task Type cliente
    End cliente;

    arrClientes: array (1..C) of cliente;

    Task Body cliente is
        Comprobante com;
    Begin
        // Implementar un entry call temporal
        Select
            Empleado.pagar(com);

```

```

        or Delay 600 // 10 * 60sec
        null // No hacer nada
    End Select
End;

Task Body empleado is
    Comprobante com;
Begin
    loop
        ACCEPT pagar(com: OUT comprobante)
        com = generarComprobante();
        End pagar;
    end loop;
End empleado;
Begin
    null;
End BancoB;

```

[Corregido]

c. Implemente una solución donde los clientes se retiran si no son atendidos inmediatamente

```

Procedure BancoC is
    Task empleado is
        Entry pagar(com: OUT comprobante);
    End empleado;

    Task Type cliente
    End cliente;

    arrClientes: array (1..C) of cliente;

    Task Body cliente is
        Comprobante com;
    Begin

```



```

        // Implementar un entry call temporal
    Select
        Empleado.pagar(com);
    else // Si no se acepta inmediatamente su pedido...
        null // No hacer nada
    End Select
End;

Task Body empleado is
    Comprobante com;
Begin
    loop
        ACCEPT pagar(com: OUT comprobante)
            com = generarComprobante();
        End pagar;
    end loop;
End empleado;
Begin
    null;
End BancoC;

```

[Corregido]

d. Implemente una solución donde los clientes esperan a lo sumo 10 minutos para ser atendidos. Si pasado ese lapso no fueron atendidos, entonces solicitan atención una vez más y se retiran si no son atendidos inmediatamente

```

Procedure BancoD is
    Task empleado is
        Entry pagar(com: OUT comprobante);
    End empleado;

    Task Type cliente
    End cliente;

```

```
arrClientes: array (1..C) of cliente;
```

```
Task Body cliente is
```

```
    Comprobante com;
```

```
Begin
```

```
    // Implementar un entry call temporal
```

```
    Select
```

```
        Empleado.pagar(com);
```

```
    or Delay 600 // 10 * 60sec
```

```
        // Intentarlo otra vez
```

```
        select
```

```
            Empleado.pagar(com);
```

```
        else // Si no se lo acepta inmediatamente
```

```
            null // No hacer nada
```

```
        End Select
```

```
    End Select
```

```
End;
```

```
Task Body empleado is
```

```
    Comprobante com;
```

```
Begin
```

```
    loop
```

```
        ACCEPT pagar(com: OUT comprobante)
```

```
        com = generarComprobante();
```

```
    End pagar;
```

```
    end loop;
```

```
End empleado;
```

```
Begin
```

```
    null;
```

```
End BancoD;
```

[Corregido]

3. Se dispone de un sistema compuesto por 1 central y 2 procesos periféricos, que se comunican

continuamente. Se requiere modelar su funcionamiento considerando las siguientes condiciones:

- **La central siempre comienza su ejecución tomando una señal del proceso 1; luego toma aleatoriamente señales de cualquiera de los dos indefinidamente. Al recibir una señal de proceso 2, recibe señales del mismo proceso durante 3 minutos.**
- **Los procesos periféricos envían señales continuamente a la central. La señal del proceso 1 será considerada vieja (se deshecha) si en 2 minutos no fue recibida. Si la señal del proceso 2 no puede ser recibida inmediatamente, entonces espera 1 minuto y vuelve a mandarla (no se deshecha).**

Procedure Punto3 is

```
// Declaraciones de tareas y de entries
```

```
Task central is
```

```
    Entry SEÑAL_INICIAL();
```

```
    Entry SEÑAL_TRABAJO_PERIF1();
```

```
    Entry SEÑAL_TRABAJO_PERIF2();
```

```
    Entry SEÑAL_CLOCK();
```

```
End central;
```

```
Task clock is
```

```
    Entry Iniciar();
```

```
End clock;
```

```
Task periferico1 is
```

```
End periferico1;
```

```
Task periferico2 is
```

```
End periferico2;
```

```

// Cuerpos

Task Body clock is
Begin
    loop
        ACCEPT Iniciar();
        DELAY 180;
        Central.SEÑAL_CLOCK();
    end loop;
End clock;

Task Body central is
    termino_tiempo: boolean;
Begin
    // La central siempre comienza su ejecución tomando una señal del
    // proceso 1
    ACCEPT SEÑAL_INICIAL
    // Luego toma aleatoriamente señales de cualquiera de los dos
    // indefinidamente
    loop
        SELECT
            ACCEPT SEÑAL_TRABAJO_PERIF1();
            // Hacer algo
        OR
            ACCEPT SEÑAL_TRABAJO_PERIF2();
            Clock.Iniciar();
            termino_tiempo:= false;
            WHILE (not termino_tiempo) DO
                SELECT
                    WHEN (SEÑAL_CLOCK'count == 0) ⇒ ACCEPT S
EÑAL_TRABAJO_PERIF2()
                OR
                    ACCEPT SEÑAL_CLOCK() DO
                        termino_tiempo:= true;
                    END SEÑAL_CLOCK;
                END SELECT;
            END WHILE;
        END SELECT;
    end loop;
End central;

```

```

                END WHILE;
            END SELECT;
        end loop;
    End central;

    Task Body periferico1 is
    Begin
        // Enviar señal inicial a la central. Utilizar una entry call simple
        Central.SEÑAL_INICIAL();

        loop
            SELECT
                Central.SEÑAL_TRABAJO_PERIF1();
            OR DELAY 120
                null;
            END SELECT;
        end loop;
    End periferico1;

    Task Body periferico2 is
    Begin
        loop
            SELECT
                Central.SEÑAL_TRABAJO_PERIF2();
            ELSE
                DELAY 60;
            END SELECT;
        end loop;
    End periferico2;

    Begin
        null;
    End Punto3;

```

[Corregido]

4. En una clínica existe un *médico* de guardia que recibe continuamente peticiones de atención de las *E enfermeras* que trabajan en su piso y de las *P personas* que llegan a la clínica ser atendidos.

Cuando una *persona* necesita que la atiendan espera a lo sumo 5 minutos a que el médico lo haga, si pasado ese tiempo no lo hace, espera 10 minutos y vuelve a requerir la atención del médico. Si no es atendida tres veces, se enoja y se retira de la clínica.

Cuando una *enfermera* requiere la atención del médico, si este no lo atiende inmediatamente le hace una nota y se la deja en el consultorio para que esta resuelva su pedido en el momento que pueda (el pedido puede ser que el médico le firme algún papel).

Cuando la petición ha sido recibida por el médico o la nota ha sido dejada en el escritorio, continúa trabajando y haciendo más peticiones.

El *médico* atiende los pedidos dándole prioridad a los enfermos que llegan para ser atendidos. Cuando atiende un pedido, recibe la solicitud y la procesa durante un cierto tiempo. Cuando está libre aprovecha a procesar las notas dejadas por las enfermeras.

```
Procedure Punto4 is
```

```
    // Declaraciones
```

```
    Task medico is
```

```
        Entry PedidoEnfermo(aux: pedido);
```

```
        Entry PedidoEnfermera();
```

```
    End medico;
```

```

Task bufferEnfermera is
    Entry DejarNota(not: IN nota);
    Entry PedirNota(not: OUT nota);
End;

```

```

Task Type enfermera is
End enfermera;

```

```

Task Type paciente is
End paciente;

```

```

arrEnfermeras: (1..E) of enfermera;
arrPacientes: (1..P) of paciente;

```

```

// Implementaciones

```

```

Task Body medico is
    aux: pedido;
    not: nota;
Begin
    loop
        SELECT
            ACCEPT PedidoEnfermo(aux) IS
                proc_sol = procesarSolicitud(aux);
                // Procesar durante un cierto tiempo
            END;
        OR
            WHEN (PedidoEnfermo'count == 0) ⇒ ACCEPT PedidoEnf
ermera
        ELSE
            // Revisar si hay alguna nota
            SELECT
                bufferEnfermera.PedirNota(not);
                ProcesarNota(not);
            ELSE

```

```

        null;
    END SELECT;
end loop;

//atiende los pedidos dándole prioridad a los enfermos
//que llegan para ser atendidos. Cuando atiende un pedido,
//recibe la solicitud y la procesa durante un cierto tiempo.
//Cuando está libre aprovecha a procesar las notas dejadas
//por las enfermeras
End medico;

// representa al consultorio
Task Body bufferEnfermera is
    colaNotas: queue<Nota>();
Begin
    loop
        // SELECT ACCEPT
        SELECT
            ACCEPT DejarNota(not: IN nota) IS
                colaNotas.push(not);
            END;
        OR
        WHEN (colaNotas.size() > 0) ⇒ ACCEPT PedirNota(not: O
UT nota) IS
            not:= pop(colaNotas);
            END;
        END SELECT;
    end loop;
End;

Task Body enfermera is
    not: nota;
Begin
    loop
        SELECT
            Medico.PedidoEnfermera();

```



```

        ELSE
            not = escribirNota();
            BufferEnfermera.DejarNota(not);
        END SELECT;
        // Trabajar
    end loop;
    // si requiere la atención del médico, si este no lo atiende inmediata
mente
    //le hace una nota y se la deja en el consultorio para que esta
    //resuelva su pedido en el momento que pueda
    //(el pedido puede ser que el médico le firme algún papel).
    //Cuando la petición ha sido recibida por el médico o la nota
    //ha sido dejada en el escritorio, continúa trabajando y
    //haciendo más peticiones
End enfermera;

Task Body paciente is
    cantIntentos: int;
    continuar: boolean;
Begin
    cantIntentos:= 0;
    continuar:= true;
    WHILE (cantIntentos < 3 && continuar) DO
        //espera a lo sumo 5 minutos a que el médico lo haga,
        SELECT
            continuar = false;
            Medico.PedidoEnfermo();
        OR DELAY 300
            cantIntentos++;
            DELAY 600
        END SELECT;
    END WHILE;
End paciente;

Begin

```

```
    null
End Punto4;
```

[Corregido]

5. En un sistema para acreditar carreras universitarias, hay UN Servidor que atiende pedidos de U Usuarios de a uno a la vez y de acuerdo con el orden en que se hacen los pedidos. Cada usuario trabaja en el documento a presentar, y luego lo envía al servidor; espera la respuesta de este que le indica si está todo bien o hay algún error. Mientras haya algún error, vuelve a trabajar con el documento y a enviarlo al servidor. Cuando el servidor le responde que está todo bien, el usuario se retira. Cuando un usuario envía un pedido espera a lo sumo 2 minutos a que sea recibido por el servidor, pasado ese tiempo espera un minuto y vuelve a intentarlo (usando el mismo documento).

Procedure Punto5 is

```
// Declaraciones
```

```
Task servidor is
```

```
    Entry Pedido(doc: IN documento; hayErrores: boolean);
```

```
End servidor;
```

```
Task Type usuario is
```

```
End usuario;
```

```
arrUsuarios: array (1..U) of usuario;
```

```
// Implementaciones
```

```

Task Body usuario is
    doc: documento;
    hayErrores: boolean;
Begin
    hayErrores:= false;

    doc = trabajar();

    WHILE (not hayErrores) DO
        SELECT
            Servidor.Pedido(doc, hayErrores);
            if (not hayErrores)
                doc = trabajar();
        OR DELAY 120
            DELAY 60;
        END SELECT;
    END WHILE;
End usuario;

Task Body servidor is
Begin
    loop
        ACCEPT Pedido(doc: IN documento; hayErrores: boolean) DO
            hayErrores:= verificarDocumento(doc);
        END Pedido;
    end loop;
End servidor;

Begin
    null
End Punto5;

```

[Corregido]

6. En una playa hay 5 equipos de 4 personas cada uno (en total son 20 personas donde cada una conoce previamente a que equipo pertenece). Cuando las personas van llegando esperan con los de su equipo hasta que el mismo esté completo (hayan llegado los 4 integrantes), a partir de ese momento el equipo comienza a jugar. El juego consiste en que cada integrante del grupo junta 15 monedas de a una en una playa (las monedas pueden ser de 1, 2 o 5 pesos) y se suman los montos de las 60 monedas conseguidas en el grupo. Al finalizar cada persona debe conocer el grupo que más dinero junto. Nota: maximizar la concurrencia. Suponga que para simular la búsqueda de una moneda por parte de una persona existe una función Moneda() que retorna el valor de la moneda encontrada.

Procedure Punto6

Task administrador is

Entry Comunicar(cant: IN int, num: IN int);

Entry ComunicarNumGrupoGanador(num_max: OUT int);

End administrador;

Task Type equipo is

Entry Llegar();

Entry Avanzar();

Entry Sumar(cant: IN int);

Entry Ident(id: IN int);

End equipo;

Task Type integrante is

End integrante;

```
arrIntegrantes: array (1..20) of integrantes;  
arrEquipos: array (1..5) of equipo;
```

Task Body integrante is

```
    num_equipo: int;  
    num_equipo_ganador: int;  
    monto_monedas: int;
```

Begin

```
    num_equipo:= X; // Ya se sabe a qué equipo pertenece  
    monto_monedas:= 0;
```

```
    // Barrera
```

```
    Equipo[num_equipo].Llegar();  
    Equipo[num_equipo].Avanzar();
```

```
    // Juntar 15 monedas
```

```
    for i:= 1 to 15 loop  
        monto_monedas:= monto_monedas + Moneda();  
    end loop;
```

```
    Equipo[num_equipo].Sumar(monto_monedas);
```

```
    Administrador.ConocerNumEquipoGanador(num_equipo_ganador);
```

End integrante;

Task Body equipo is

```
    num_equipo: int;  
    monto: int;  
    parcial: int;
```

Begin

```
    monto:= 0;  
    ACCEPT Ident(id: IN int) DO  
        num_equipo:= id;  
    END Ident;
```

```

    for i:= 1 to 4 loop
        ACCEPT Llegar();
    end loop;

    for i:= 1 to 4 loop
        ACCEPT Avanzar();
    end loop;

    for i:= 1 to 4 loop
        ACCEPT Sumar(parcial);
        monto:= monto + parcial;
    end loop;

    Administrador.Comunicar(monto, num_equipo);
End equipo;

Task Body administrador is
    monto: int;
    monto_max: int;
    num: int;
    num_max: int;
Begin
    for i:= 1 to 5 loop
        ACCEPT Comunicar(cant: IN int, num: IN int) DO
            if (cant > monto_max) then
                monto_max:= cant;
                num_max:= num;
            end if;
        END Comunicar;
    end loop;

    for i:= 1 to 20 loop
        ACCEPT ConocerNumGrupoGanador(num_max: OUT int) DO
            num_max:= num_max;
        END ConocerNumGrupoGanador;
    end loop;

```

```

    End;
Begin
    for i:= 1 to 5 loop
        equipo[i].Ident(i);
    end loop;
End Punto6;

```

[Corregido]

7. Se debe calcular el valor promedio de un vector de 1 millón de números enteros que se encuentra distribuido entre 10 procesos Worker (es decir, cada Worker tiene un vector de 100 mil números). Para ello, existe un Coordinador que determina el momento en que se debe realizar el cálculo de este promedio y que, además, se queda con el resultado. Nota: maximizar la concurrencia; este cálculo se hace una sola vez.

Procedure Punto7

```

Task Coordinador is
    Entry Llegar();
    Entry Terminar(cant: IN int);
End Coordinador;

Task Type Worker is
    Entry Avanzar();
End Worker;

arrWorkers: array (1..10) of Worker;

// Implementaciones

Task Body Worker is

```

```

    parte_vector: int[100.000];
    suma: int;
Begin
    suma:= 0;

    // El coordinador determina el momento en que se debe
    // realizar el cálculo de este promedio
    Coordinador.Llegar();
    ACCEPT Avanzar();

    for i:= 1 to 100.000 loop
        suma:= suma + parte_vector[i];
    end loop;

    // Sacar promedio
    suma:= suma / 100.000;

    Coordinador.Terminar(suma);
End;

Task Body Coordinador is
    suma: int;
Begin
    suma:= 0;

    for i:= 1 to 10 loop
        ACCEPT Llegar;
    end loop;

    for i:= 1 to 10 loop
        arrWorkers[i].Avanzar();
    end loop;

    for i:= 1 to 10 loop
        ACCEPT Terminar(cant: IN int) DO
            suma:= suma + cant;

```



```
END Terminar;  
end loop;  
End Coordinador;
```

```
Begin  
End Punto7;
```

[Corregido]

8. Hay un sistema de reconocimiento de huellas dactilares de la policía que tiene 8 Servidores para realizar el reconocimiento, cada uno de ellos trabajando con una Base de Datos propia; a su vez hay un Especialista que utiliza indefinidamente. El sistema funciona de la siguiente manera: el Especialista toma una imagen de una huella (TEST) y se la envía a los servidores para que cada uno de ellos le devuelva el código y el valor de similitud de la huella que más se asemeja a TEST en su BD; al final del procesamiento, el especialista debe conocer el código de la huella con mayor valor de similitud entre las devueltas por los 8 servidores. Cuando ha terminado de procesar una huella comienza nuevamente todo el ciclo. Nota: suponga que existe una función Buscar(test, código, valor) que utiliza cada Servidor donde recibe como parámetro de entrada la huella test, y devuelve como parámetros de salida el código y el valor de similitud de la huella más parecida a test en la BD correspondiente. Maximizar la concurrencia y no generar demora innecesaria.

Procedure Punto8 is

Task Especialista is

Entry EnviarImagen(huella: IN imagen);

Entry PedirImagen(huella: OUT imagen);

Entry RecibirInformacion(codigo: IN int, valorsimil: IN float);

Entry Siguiente();

End Especialista;

Task Type Servidor is

End Servidor;

arrServidores: array (1..8) of Servidor;

Task Body Especialista is

huella: imagen;

codigo_max: int;

valorsimil_max: float;

Begin

loop

huella = tomarImagen();

codigo_max:= 0;

valorsimil_max:= -1;

for i:= 1 to 16 loop

SELECT

ACCEPT EnviarImagen(huella: OUT imagen) DO

huella:= huella;

END EnviarImagen;

OR

ACCEPT RecibirInformacion(codigo: IN int; valorsimil: I

N float) DO

if (valorsimil > valorsimil_max) then

valorsimil_max:= valorsimil;

codigo_max:= codigo;

end if;

```

        END RecibirInformacion;
    END SELECT;
end loop;

    // Cuando ha terminado de procesar una
    // huella comienza nuevamente todo el ciclo → Consultar esta
parte de nuevo
    for i:= 1 to 8 loop
        ACCEPT Siguiente();
    end loop;
end loop;
End Especialista;

Task Body Servidor is
    huella: imagen;
    codigo: int;
    valorsimil: float;
Begin
    loop
        Especialista.PedidolImagen(huella);
        Buscar(huella, codigo, valorsimil);
        Especialista.RecibirInformacion(codigo, valorsimil);
        Especialista.Siguiente();
    end loop;
End Servidor;
Begin
    null;
End Punto8;

```

[Corregido]

9. Una empresa de limpieza se encarga de recolectar residuos en una ciudad por medio de 3 camiones. Hay P personas que hacen reclamos continuamente hasta que uno de los camiones pase por su casa. Cada

persona hace un reclamo y espera a lo sumo 15 minutos a que llegue un camión; si no pasa, vuelve a hacer el reclamo y a esperar a lo sumo 15 minutos a que llegue un camión; y así sucesivamente hasta que el camión llegue y recolecte los residuos. Sólo cuando un camión llega, es cuando deja de hacer reclamos y se retira. Cuando un camión está libre la empresa lo envía a la casa de la persona que más reclamos ha hecho sin ser atendido. Nota: maximizar la concurrencia

Procedure Punto9 is

Task Coordinador is

Entry Aviso(id: IN int);

Entry Reclamar(id: IN int);

Entry RecibirDestino(id_dest: OUT int);

End Coordinador;

Task Type Persona is

Entry Ident(id: IN int);

Entry LlegarCamion();

End Persona;

Task Type Camion is

End Camion;

arrPersonas: array (1..P) of Persona;

arrCamiones: array (1..3) of Camion;

Task Type Persona is

id: int;

llego_camion: boolean;;

Begin

llego_camion:= false;

```

ACCEPT Ident(id: IN int) DO
    id:= id;
END Ident;

Coordinador.Aviso(id);
WHILE (not Llego_camion) DO
    SELECT
        ACCEPT LlegarCamion();
        Llego_camion:= true;
    OR DELAY 900
        Coordinador.Reclamar(id);
    END SELECT;
END WHILE;
End Persona;

Task Type Coordinador is
    personas_esperando: int;
    cantidad_esperas: int[P];
Begin
    personas_esperando:= 0;

    loop
        SELECT
            ACCEPT Aviso(id: IN int) DO
                personas_esperando++;
                cantidad_esperas[id] = 0;
            END Aviso;
        OR
            ACCEPT Reclamar(id: IN int) DO
                // No sé si esto está bien, pero el ayudante me
                // había dicho que había que tener alguna forma
                // de ignorar los reclamos de alguien que
                // ya tuviera un camión yendo en su dirección
                if (cantidad_esperas[id] <> -1) then
                    cantidad_esperas[id]++;

```

```

        end then;
    END Reclamar;
OR
    WHEN (personas_esperando > 0) ⇒ ACCEPT RecibirDestino
o(id: OUT int) DO
        id:= max(cantidad_espera);
        cantidad_espera[max(cantidad_espera)] = -1;
    END RecibirDestino;
END SELECT;
end loop;
End Coordinador;

Task Type Camion is
    id_destino: int;
Begin
    loop
        Coordinador.RecibirDestino(id_destino);
        Persona[id_destino].LlegarCamion();
    end loop;
End Camion;

Begin
    for i:= 1 to P loop
        Persona.Ident(i);
    end loop;
End Punto9;

```

[[Consultar](#)]